Introduction to transcriptomics - practical

- conda create -n hisat2_env hisat2

This will install the latest version of hisat2 available on anaconda in the environment 'hisat2_env'. To activate and use hisat2 we now do:

- source activate hisat2_env

We can repeat this for any software we need to use when aligning RNAseq reads to a reference genome, and downstream analyses.

The next thing to do is to make your reference genome ready for alignment. We are going to use the same reference we used on Day 1. This is the most up-to-date reference genome for *C. auris* – there are available reference genomes for many species, which you can find on NCBI GenBank and download. Periodically, new version are released, which provide improved resolution.

We will now index the reference genome. Firstly, we extract splice sites and exons using the gtf file (also available on GenBank):

hisat2_extract_splice_sites.py GCA_002759435.2_Cand_auris_B8441_V2_genomic.gtf > GCA_002759435.2_Cand_auris_B8441_V2_genomic.ss

hisat2_extract_exons.py GCA_002759435.2_Cand_auris_B8441_V2_genomic.gtf > GCA_002759435.2_Cand_auris_B8441_V2_genomic.exon

Now we will build the index for HISAT2:

hisat2-build -p 16 --ss GCA_002759435.2_Cand_auris_B8441_V2_genomic.ss --exon GCA_002759435.2_Cand_auris_B8441_V2_genomic.exon GCA_002759435.2_Cand_auris_B8441_V2_genomic.fa genome

Now we are ready to align!

HISAT2 basic usage is as follows:

- hisat2 [option]* -x <ht2-idx> {-1 <m1> -2 <m2> | -U <r> | [-S <sam>]

However it is really beneficial to use some of the optional extras:

- '-p 8' tells HISAT2 to use eight CPUs for bowtie alignments.
- '–rna-strandness RF' specifies strandness of RNAseq library. We will specify RF since the TruSeq strand-specific library was used to make these libraries. See here for options.
- '–rg-id $ID' specifies a read group ID that is a unique identifier.

- ’–rg SM:$SAMPLE_NAME’ specifies a read group sample name. This together with rg-id will allow you to determine which reads came from which sample in the merged bam later on.
- ’–rg LB:$LIBRARY_NAME’ specifies a read group library name. This together with rg-id will allow you to determine which reads came from which library in the merged bam later on.
- ’–rg PL:ILLUMINA’ specifies a read group sequencing platform.
- ’–rg PU:$PLATFORM_UNIT’ specifies a read group sequencing platform unit. Typically this consists of FLOWCELL-BARCODE.LANE
- ’–dta’ Reports alignments tailored for transcript assemblers.
- ‘-x /path/to/hisat2/index’ The HISAT2 index filename prefix (minus the trailing .X.ht2) built earlier including splice sites and exons.
- ‘-1 /path/to/read1.fastq.gz’ The read 1 FASTQ file, optionally gzip(.gz) or bzip2(.bz2) compressed.
- ‘-2 /path/to/read2.fastq.gz’ The read 2 FASTQ file, optionally gzip(.gz) or bzip2(.bz2) compressed.
- ‘-S /path/to/output.sam’ The output SAM format text file of alignments.

We only need a few of these options today:

rg-sm = PR_0h_REP1
rg-id = VH00344
rg-pl = ILLUMINA
rg-lb = rnaseq
rg-pu = AAAKMY2M5

- Can you tell where these options come from?

Let’s align!

Type the following (adjust for your specific file names and paths):

```
hisat2 -p 8 --rg-id=VH00344 --rg SM:PR_0h_REP1 --rg LB:rnaseq --rg PL:illumina --rg PU:AAKMY2M5 -x /path/to/reference.fasta --dta --rna-strandness RF -1 /path/to/file1.fastq.gz -2 /path/to/file2.fastq.gz -S /path/to/output_dir/PR_0h_REP1.sam
```

Next we need to convert to a BAM file. SAM files can be very large, it is imperative to convert these files to BAM format, which is a compressed version of the SAM format, in order to save space.

Create a samtools environment using conda (as you did for hisat2)

```
samtools view -u /path/to/output_dir/PR_0h_REP1.sam | samtools sort -@ 8 - /path/to/output_dir/PR_0h_REP1
```

You now have an alignment! At this stage you can assess the quality of your alignment (using samtools flagstat, for example). FastQC also provides a html report after performing very basic QC of the BAM file. Picard will generate RNAseq-specific quality metrics and figures, but

this would require indexing of the reference genome file (again!) – if you're interested, we can go over this later. There are also many tools freely available on github.

We will use Stringtie (create a new conda environment) to estimate expression from the BAM file generated by HISAT2. Stringtie has an advantage over other software as it can be used to align *de novo* assembled fragments that combine pairs of reads into longer sequences, thus improving accuracy. This provides a better transcriptome, as it can create/identify new transcripts, which pairs with the splice-aware aligner HISAT2 well.

Basic Stringtie usage:

stringtie <aligned_reads.bam> [options]*

We will use a few more options!

stringtie --rf -B -p 28 -j 3 -c 7.5 -g 1 -v -G /path/to/reference.gtf -e -o /path/to /output/transcripts.gtf -A /path/to/output/abundances.csv /path/to/output/ PR_0h_REP1.bam

The options we've used are as follows:

--rf tells StringTie that our data is stranded and to use the correct strand specific mode
-o output file name for assembled transcripts
-A output file name for gene abundance estimates
-p is the number of CPUs to use
-v verbose output
-G gtf/gff3 file
-B is to create the Ballgown specific output files in the same directory as all the other files (-b is to create the files in a different directory)

It's important to note that we are running our alignment on just one replicate – running on all three in one session would take too long. But we move on to finding DEGs using some data I made earlier….

Using Stringtie we merge all of our data now (I'll provide you with the extra replicates):

stringtie --merge PR_0h_REP1/transcripts.gtf PR_0h_REP2/transcripts.gtf PR_0h_REP3/transcripts.gtf -o PR_0h.merged.gtf -G /path/to/reference/GCA_002759435.2_Cand_auris_B8441_V2_genomic.no_genes.gtf

This generates files ending with 'ctab':

- e_data.ctab: exon-level expression measurements. One row per exon. Columns are e_id (numeric exon id), chr, strand, start, end (genomic location of the exon), and the following expression measurements for each sample:
    - rcount: reads overlapping the exon
    - ucount: uniquely mapped reads overlapping the exon

    - mrcount: multi-map-corrected number of reads overlapping the exon
    - cov average per-base read coverage
    - cov_sd: standard deviation of per-base read coverage
    - mcov: multi-map-corrected average per-base read coverage
    - mcov_sd: standard deviation of multi-map-corrected per-base coverage
- i_data.ctab: intron- (i.e., junction-) level expression measurements. One row per intron. Columns are i_id(numeric intron id), chr, strand, start, end (genomic location of the intron), and the following expression measurements for each sample:
    - rcount: number of reads supporting the intron
    - ucount: number of uniquely mapped reads supporting the intron
    - mrcount: multi-map-corrected number of reads supporting the intron
- t_data.ctab: transcript-level expression measurements. One row per transcript. Columns are:
    - t_id: numeric transcript id
    - chr, strand, start, end: genomic location of the transcript
    - t_name: Cufflinks-generated transcript id
    - num_exons: number of exons comprising the transcript
    - length: transcript length, including both exons and introns
    - gene_id: gene the transcript belongs to
    - gene_name: HUGO gene name for the transcript, if known
    - cov: per-base coverage for the transcript (available for each sample)
    - FPKM: Cufflinks-estimated FPKM for the transcript (available for each sample)
- e2t.ctab: table with two columns, e_id and t_id, denoting which exons belong to which transcripts. These ids match the ids in the e_data and t_data tables.
- i2t.ctab: table with two columns, i_id and t_id, denoting which introns belong to which transcripts. These ids match the ids in the i_data and t_data tables.

Now we move onto ballgown in R/RStudio

<u>Differential expression analysis</u>

We will use ballgown in R, which can now be run on your personal laptop.

Ballgown is a software package designed to facilitate flexible differential expression analysis of RNAseq data, but primarily for reference-based assemblies.

Open RStudio (I prefer this, as the layout is user friendly and you can visualise plots instantly, as well as review previous code) and install ballgrown:

```
if (!require("BiocManager", quietly = TRUE))
install.packages("BiocManager")

BiocManager::install("ballgown")
```

We need to do a bit of organising, and put all of the ctab files into folders like this:

```
Jo Rhodes@Johannas-MacBook-Pro Desktop % ls -lh extdata
total 0
drwxr-xr-x  7 Jo Rhodes  staff   224B  2 Sep 16:06 PR_0h_REP1
drwxr-xr-x  7 Jo Rhodes  staff   224B  2 Sep 16:06 PR_0h_REP2
drwxr-xr-x  7 Jo Rhodes  staff   224B  2 Sep 16:07 PR_0h_REP3
drwxr-xr-x  7 Jo Rhodes  staff   224B  2 Sep 16:07 PR_6h_REP1
drwxr-xr-x  7 Jo Rhodes  staff   224B  2 Sep 16:07 PR_6h_REP2
drwxr-xr-x  7 Jo Rhodes  staff   224B  2 Sep 16:07 PR_6h_REP3
drwxr-xr-x  7 Jo Rhodes  staff   224B  2 Sep 16:08 SUS_0h_REP1
drwxr-xr-x  7 Jo Rhodes  staff   224B  2 Sep 16:08 SUS_0h_REP2
drwxr-xr-x  7 Jo Rhodes  staff   224B  2 Sep 16:08 SUS_0h_REP3
drwxr-xr-x  7 Jo Rhodes  staff   224B  2 Sep 16:09 SUS_6h_REP1
drwxr-xr-x  7 Jo Rhodes  staff   224B  2 Sep 16:09 SUS_6h_REP2
drwxr-xr-x  7 Jo Rhodes  staff   224B  2 Sep 16:09 SUS_6h_REP3
Jo Rhodes@Johannas-MacBook-Pro Desktop % ls -lh extdata/PR_0h_REP1
total 2336
-rw-rw----@ 1 Jo Rhodes  staff    57K  2 Sep 15:53 e2t.ctab
-rw-rw----@ 1 Jo Rhodes  staff   475K  2 Sep 15:53 e_data.ctab
-rw-rw----@ 1 Jo Rhodes  staff   3.9K  2 Sep 15:53 i2t.ctab
-rw-rw----@ 1 Jo Rhodes  staff    21K  2 Sep 15:53 i_data.ctab
-rw-rw----@ 1 Jo Rhodes  staff   603K  2 Sep 15:53 t_data.ctab
Jo Rhodes@Johannas-MacBook-Pro Desktop %
```

Then we make a comma separated file (I made one called 'pheno_PR.csv') to tell ballgown where the data are. For now, let's compare PR_0h and PR_6h (this is a resistant isolate at 0h and 6h after exposure to drug – I'll provide you with the extra data). To do this, either use Excel and save as a csv file, or you can use terminal (with the 'vi' command).

```
Jo Rhodes@Johannas-MacBook-Pro PR0h_vs_6h % more pheno_PR.csv
ids,type,path
PR_0h_REP1,PR_0h,extdata/PR_0h_REP1
PR_0h_REP2,PR_0h,extdata/PR_0h_REP2
PR_0h_REP3,PR_0h,extdata/PR_0h_REP3
PR_6h_REP1,PR_6h,extdata/PR_6h_REP1
PR_6h_REP2,PR_6h,extdata/PR_6h_REP2
PR_6h_REP3,PR_6h,extdata/PR_6h_REP3
```

In RStudio, we will load the necessary libraries (install packages if you haven't already):

library(ballgown)
library(genefilter)
library(dplyr)
library(devtools)

Navigate in RStudio to the directory where the csv file is saved and load:

- pheno_data_PR = read.csv("pheno_PR.csv")

Load ballgown data structure and save it to a variable ‘bg’:

- bg = ballgown(samples=as.vector(pheno_data_PR$path), pData=pheno_data_PR)

Display a description of the bg object. You can also save this to a file for later use (‘save(bg, file=bg_PR.rda’)

Now filter low abundance genes/transcripts by removing all transcripts with a variance across the samples of less than one:

- bg_filt <- subset(bg, "rowVars(texpr(bg)) >1", genomesubset=TRUE)

Load all attributes, including gene name:

- bg_table = texpr(bg_filt, 'all')
- bg_gene_names = unique(bg_table[, 9:10])

Again, this can be saved for later use (‘save(bg_filt, file='bg_filt_SUS.rda')’). Now perform your differential analysis using the filtered data:

- results_transcripts = stattest(bg_filt, feature="transcript", covariate="type", getFC=TRUE, meas="FPKM")
- results_genes = stattest(bg_filt, feature="gene", covariate="type", getFC=TRUE, meas="FPKM")
- results_genes = merge(results_genes, bg_gene_names, by.x=c("id"), by.y=c("gene_id"))

You can now output all of the lists and save to files:

- write.table(results_transcripts, "PR0h_vs_6h_transcript_results.tsv", sep="\t", quote=FALSE, row.names = FALSE)
- write.table(results_genes, "PRS0h_vs_6h_gene_results.tsv", sep="\t", quote=FALSE, row.names = FALSE)

To identify significant genes with a p-value <0.05 and/or identify logFC we can do the following (and again, save to files):

- results_transcripts$logFC=log2(results_transcripts$fc) ## logFC
- results_genes$logFC=log2(results_genes$fc) ## logFC
- sig_genes=subset(results_genes,results_genes$pval<0.05) ## significant genes
- sig_transcripts=subset(results_transcripts,results_transcripts$pval<0.05) ## significant transcripts
- write.table(sig_genes, "SUS0h_vs_6h_gene_results_sig.tsv", sep="\t", quote=FALSE, row.names = FALSE)
- write.table(sig_transcripts, "SUS0h_vs_6h_transcripts_results_sig.tsv", sep="\t", quote=FALSE, row.names = FALSE)

- gene_Expression=gexpr(bg_filt)
- write.csv(gene_Expression,"SUS0h_vs_6h.csv",row.names=FALSE)
- write.csv(merge(results_genes, gene_Expression, by=0, all=T), "all_results.csv", row.names=TRUE)

Differentially expressed genes are those defined as having a p-value < 0.05. This can obviously be changed to be more stringent. A lot of studies also look into log fold change (logFC), which is another cutoff for differential expression. This is a transformation at base 2, so the interpretation is straightforward: a log(FC) of 1 means it is twice as expressed. Without the log, a downregulation gets squeezed between 0 and 1, whilst an upregulation goes from 1 to infinity; with the log transformation, you get a better scale (centred around zero and symmetrical). However, again, the cutoff is usually down to the user! Sometimes you can look at what other people have done in the same organism, or you can plot the fold change values, and pick a cutoff yourself. For this *C. auris* dataset, I used logFC > 0.6 as upregulated and logFC < -0.6 as downregulated.

All the files we've created can be opened in Excel or similar.

Answer the following questions:

1. How many significant genes do you have?
2. What are the top 20 up- and downregulated genes?

Visualisation is a whole extra topic! There are many ways to visualise your data. I recommend looking into ggplot2 in R. The following tutorials are also worth a look:

Visualisation of volcano plots: https://samdsblog.netlify.app/post/visualizing-volcano-plots-in-r/
Description of volcano plots relating to DEGs:
https://biocorecrg.github.io/CRG_RIntroduction/volcano-plots.html
Tutorials on RNAseq, plus basic plots in ggplot2:
https://pydupont.github.io/BallGownTuto/Protocol.html

ADVANCED: *De novo* assembly

Trinity is software used to perform *de novo* transcriptome assembly. The current version on the HPC is out-of-date, so create a conda environment with the most up-to-date version (v2.13.2).

Sometimes spaces are included in the headers (see description of what a fastq file looks like earlier in this document), which Trinity cannot handle and will fail at *in silico* normalisation. To remove these, first copy your fastq files (just in case something goes wrong. This way you don't lose all your data!):

- cp filename > new_filename.fastq.gz

Then type the following:

```
for i in new_filename.fastq.gz*
do
zcat $i | sed '/^@/ s/ .*//' > trinity_1_$i
done
```

This deleted everything after the space in the header. The next step then includes the "/1" at the end of the headers (swap for "/2" for R2 file):

```
for i in trinity_1_*
do
awk '{ if (NR%4==1) { print $1""$2"/1" } else { print } }' $i  > new_$i
done
```

You can then run Trinity as follows, which includes trimmomatic to trim low quality reads and *in silico* normalisation:

```
Trinity --seqType fq --max_memory 50G --left $output/R1.fastq.gz --right $output/R2.fastq.gz
--CPU 28 --output $output --trimmomatic --min_kmer_cov 2 --normalize_by_read_set
```

It’s important to assess the quality of your assembly. Trinity includes perl scripts to assess this:

```
TrinityStats.pl Trinity.fasta
```

This will provide details on the counts of transcripts, N50, average contig length and total assembled bases. I would also advise computing BUSCO to explore the completeness of the assembly according to conserved ortholog content.